

ALGORITHMS & DATA STRUCTURES

HANDS ON — Complete Solutions

Submitted by:

Student Name : DEVANSH SAHU

Email id : devanshstar7@gmail.com

Superset ID : 7797591

B.Tech Computer Science & Engineering | Java Programming Lab

Contents Overview

Exercise 1 — Inventory Management System (HashMap)

Exercise 2 — E-commerce Search Function (Linear & Binary Search)

Exercise 3 — Customer Order Sorting (Bubble & Quick Sort)

Exercise 4 — Employee Management System (Array)

Exercise 5 — Task Management System (Singly Linked List)

Exercise 6 — Library Management System (Search Algorithms)

Exercise 7 — Financial Forecasting (Recursive Algorithms)

Exercise 1: Inventory Management System

► Scenario Understanding

You are developing an inventory management system for a warehouse. Efficient data storage and retrieval are crucial for handling thousands of SKUs with fast add, update, and delete operations.

► Why Data Structures & Algorithms Are Essential

- A warehouse may store thousands or millions of products — sequential scan ($O(n)$) is too slow.
- HashMap provides $O(1)$ average-case lookup by productId — critical for real-time warehouse operations.
- Choosing the wrong data structure leads to bottlenecks: e.g., ArrayList requires $O(n)$ search by ID.
- Algorithms ensure correct ordering, deduplication, and integrity of inventory records.

► Suitable Data Structures

- HashMap<Integer, Product> — Best for this problem: $O(1)$ add/get/delete by key (productId).
- ArrayList<Product> — Simple but $O(n)$ search; better for ordered traversal.
- TreeMap<Integer, Product> — Sorted keys, $O(\log n)$ operations; useful for range queries.

► Java Implementation

```
class Product {
    private int productId;
    private String productName;
    private int quantity;
    private double price;

    public Product(int productId, String productName, int quantity, double price) {
        this.productId = productId;
        this.productName = productName;
        this.quantity = quantity;
        this.price = price;
    }
    // Getters and setters omitted for brevity
    @Override
    public String toString() {
        return String.format("Product[ID=%d, Name=%s, Qty=%d, Price=%.2f]",
            productId, productName, quantity, price);
    }
}

class InventoryManagement {
    private Map<Integer, Product> inventory = new HashMap<>();

    public void addProduct(Product product) { // O(1) avg
        if (inventory.containsKey(product.getProductId())) {
            System.out.println("Product already exists.");
            return;
        }
    }
}
```

```

    }
    inventory.put(product.getProductId(), product);
}

public void updateProduct(int id, int qty, double price) { // O(1) avg
    Product p = inventory.get(id);
    if (p == null) { System.out.println("Not found."); return; }
    p.setQuantity(qty); p.setPrice(price);
}

public void deleteProduct(int id) { // O(1) avg
    inventory.remove(id);
}
}

```

► Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
add()	$O(1)$	$O(1)$	$O(n) *$	$O(n)$
update()	$O(1)$	$O(1)$	$O(n) *$	$O(1)$
delete()	$O(1)$	$O(1)$	$O(n) *$	$O(1)$
displayAll()	$O(n)$	$O(n)$	$O(n)$	$O(1)$

* Worst case $O(n)$ occurs only during hash collision rehashing — extremely rare with good hash function.

► Output Screenshot

```
Exercise 1 – Output Terminal

=====
Exercise 1: Inventory Management
=====

Added: Product[ID=101, Name=Laptop, Qty=50, Price=75000.00]
Added: Product[ID=102, Name=Mouse, Qty=200, Price=599.00]
Added: Product[ID=103, Name=Keyboard, Qty=150, Price=999.00]
Added: Product[ID=104, Name=Monitor, Qty=80, Price=12000.00]

--- Current Inventory ---
Product[ID=101, Name=Laptop, Qty=50, Price=75000.00]
Product[ID=102, Name=Mouse, Qty=200, Price=599.00]
Product[ID=103, Name=Keyboard, Qty=150, Price=999.00]
Product[ID=104, Name=Monitor, Qty=80, Price=12000.00]
-----

-- Updating Product 102 --
Updated: Product[ID=102, Name=Mouse, Qty=180, Price=649.00]

-- Deleting Product 104 --
Deleted: Product[ID=104, Name=Monitor, Qty=80, Price=12000.00]

-- Trying duplicate add --
Product with ID 101 already exists.

--- Current Inventory ---
Product[ID=101, Name=Laptop, Qty=50, Price=75000.00]
Product[ID=102, Name=Mouse, Qty=180, Price=649.00]
Product[ID=103, Name=Keyboard, Qty=150, Price=999.00]
-----

--- Time Complexity Analysis ---
Add      : O(1) average, O(n) worst (hash collision)
Update   : O(1) average
Delete   : O(1) average
Display  : O(n)
```

Exercise 2: E-commerce Platform Search Function

► Big O Notation Explained

Big O notation describes the upper-bound growth rate of an algorithm's time or space requirements as input size n grows. It abstracts away machine-specific constants and focuses on algorithmic efficiency.

- $O(1)$ — Constant: Execution time does not change with input size. Example: HashMap lookup.
- $O(\log n)$ — Logarithmic: Input halved each step. Example: Binary Search.
- $O(n)$ — Linear: Proportional to input. Example: Linear Search.
- $O(n^2)$ — Quadratic: Nested loops. Example: Bubble Sort.

► Search Scenarios

- Best Case: Target is the first element (Linear) or middle element (Binary) — $O(1)$.
- Average Case: Linear $O(n/2) \approx O(n)$; Binary $O(\log n)$.
- Worst Case: Target not found — Linear $O(n)$; Binary $O(\log n)$.

► Java Implementation

```
class Product {
    private int productId;
    private String productName;
    private String category;
    // constructor + getters
}

// Linear Search - O(n)
public static Product linearSearch(Product[] products, String targetName) {
    for (Product p : products) {
        if (p.getProductName().equalsIgnoreCase(targetName))
            return p;
    }
    return null;
}

// Binary Search - O(log n) - requires sorted array
public static Product binarySearch(Product[] sorted, String targetName) {
    int low = 0, high = sorted.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        int cmp = sorted[mid].getProductName().compareToIgnoreCase(targetName);
        if (cmp == 0) return sorted[mid];
        else if (cmp < 0) low = mid + 1;
        else high = mid - 1;
    }
    return null;
}
```

► Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
Linear Search	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Sort (Arrays)	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(\log n)$

Recommendation: For the e-commerce platform with large sorted product catalogs, Binary Search is significantly more efficient.

► Output Screenshot

```

=====
Exercise 2: E-commerce Search Function
=====

Product Catalog:
SearchProduct[ID=1, Name=Laptop           , Category=Electronics]
SearchProduct[ID=2, Name=Backpack         , Category=Accessories]
SearchProduct[ID=3, Name=Headphones       , Category=Electronics]
SearchProduct[ID=4, Name=Notebook         , Category=Stationery]
SearchProduct[ID=5, Name=Smartphone       , Category=Electronics]
SearchProduct[ID=6, Name=Watch            , Category=Accessories]
SearchProduct[ID=7, Name=Tablet           , Category=Electronics]

--- Linear Search ---
Searching for: Headphones
Result: SearchProduct[ID=3, Name=Headphones, Category=Electronics]
Time: 3106 ns

--- Binary Search (Sorted Array) ---
Searching for: Headphones
Result: SearchProduct[ID=3, Name=Headphones, Category=Electronics]
Time: 3732 ns

--- Searching Non-existent Product ---
Linear: None    Binary: None

--- Big O Analysis ---
Linear Search : Best  $O(1)$  | Avg  $O(n)$  | Worst  $O(n)$ 
Binary Search : Best  $O(1)$  | Avg  $O(\log n)$  | Worst  $O(\log n)$ 
Conclusion    : Binary Search preferred for large sorted datasets.

```

Exercise 3: Sorting Customer Orders

► Sorting Algorithms Overview

- Bubble Sort: Repeatedly swaps adjacent elements if out of order. Simple but inefficient for large datasets.
- Insertion Sort: Builds sorted portion one element at a time. Efficient for nearly-sorted data — $O(n)$ best case.
- Quick Sort: Divide-and-conquer using a pivot. $O(n \log n)$ average — industry standard.
- Merge Sort: Divides array recursively, merges sorted halves. Stable, $O(n \log n)$ guaranteed.

► Java Implementation

```
class Order {
    private int orderId;
    private String customerName;
    private double totalPrice;
    // constructor + getters
}

// Bubble Sort -  $O(n^2)$ 
public static void bubbleSort(Order[] orders) {
    int n = orders.length;
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (orders[j].getTotalPrice() > orders[j+1].getTotalPrice()) {
                Order temp = orders[j];
                orders[j] = orders[j+1];
                orders[j+1] = temp;
            }
        }
    }
}

// Quick Sort -  $O(n \log n)$  average
public static void quickSort(Order[] orders, int low, int high) {
    if (low < high) {
        int pi = partition(orders, low, high);
        quickSort(orders, low, pi - 1);
        quickSort(orders, pi + 1, high);
    }
}

private static int partition(Order[] orders, int low, int high) {
    double pivot = orders[high].getTotalPrice();
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (orders[j].getTotalPrice() <= pivot) {
            i++;
            Order temp = orders[i]; orders[i] = orders[j]; orders[j] = temp;
        }
    }
    Order temp = orders[i+1]; orders[i+1] = orders[high]; orders[high] = temp;
    return i + 1;
}
```

}

► Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$
Insert Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$

Why Quick Sort is preferred: Average $O(n \log n)$ with low constant factors and in-place sorting ($O(\log n)$ space). Bubble Sort becomes impractical for $n > 10,000$ orders.

► Output Screenshot

```

=====
Exercise 3: Sorting Customer Orders
=====

Original Orders:
Order[ID=1001, Customer=Alice           , Price=4500.00]
Order[ID=1002, Customer=Bob             , Price=1200.50]
Order[ID=1003, Customer=Charlie         , Price=9800.75]
Order[ID=1004, Customer=Diana           , Price=300.00]
Order[ID=1005, Customer=Edward          , Price=7650.25]

--- After Bubble Sort (ascending by price) ---
Order[ID=1004, Customer=Diana           , Price=300.00]
Order[ID=1007, Customer=George          , Price=550.80]
Order[ID=1002, Customer=Bob             , Price=1200.50]
Order[ID=1006, Customer=Fiona           , Price=2250.00]
Order[ID=1001, Customer=Alice           , Price=4500.00]
Order[ID=1005, Customer=Edward          , Price=7650.25]
Order[ID=1003, Customer=Charlie         , Price=9800.75]
Bubble Sort Time: 6172 ns

--- After Quick Sort (ascending by price) ---
Order[ID=1004, Customer=Diana           , Price=300.00]
Order[ID=1002, Customer=Bob             , Price=1200.50]
Order[ID=1001, Customer=Alice           , Price=4500.00]
Order[ID=1005, Customer=Edward          , Price=7650.25]
Order[ID=1003, Customer=Charlie         , Price=9800.75]
Quick Sort Time: 7999 ns

--- Performance Comparison ---
Algorithm   | Best      | Average   | Worst     | Space
-----|-----|-----|-----|-----
Bubble Sort |  $O(n)$     |  $O(n^2)$    |  $O(n^2)$    |  $O(1)$ 
Quick Sort  |  $O(n \log n)$  |  $O(n \log n)$  |  $O(n^2)$    |  $O(\log n)$ 

Conclusion: Quick Sort is preferred for large datasets.

```


Exercise 4: Employee Management System

► Array Representation in Memory

Arrays are contiguous blocks of memory where elements are stored sequentially. Each element is accessed via its index: $\text{address} = \text{base_address} + (\text{index} \times \text{element_size})$. This gives $O(1)$ random access.

- Advantages: $O(1)$ index access, cache-friendly memory layout, simple and predictable.
- Disadvantages: Fixed size, $O(n)$ insertion/deletion (requires shifting), costly resizing.
- Best used when: Dataset size is known, random access needed, insertions/deletions rare.

► Java Implementation

```
class Employee {
    private int employeeId;
    private String name;
    private String position;
    private double salary;
    // constructor, getters, toString()
}

class EmployeeArray {
    private Employee[] employees;
    private int size, capacity;

    public EmployeeArray(int capacity) {
        this.capacity = capacity;
        this.employees = new Employee[capacity];
        this.size = 0;
    }

    // Add - O(1) when capacity available
    public boolean addEmployee(Employee emp) {
        if (size >= capacity) { System.out.println("Array full!"); return false; }
        employees[size++] = emp;
        return true;
    }

    // Search - O(n) linear scan
    public Employee searchEmployee(int id) {
        for (int i = 0; i < size; i++)
            if (employees[i].getEmployeeId() == id) return employees[i];
        return null;
    }

    // Delete - O(n): scan + shift remaining elements
    public boolean deleteEmployee(int id) {
        for (int i = 0; i < size; i++) {
            if (employees[i].getEmployeeId() == id) {
                for (int j = i; j < size - 1; j++)
                    employees[j] = employees[j+1];
                employees[--size] = null;
                return true;
            }
        }
    }
}
```

```

    }
    return false;
}
}

```

► Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
<code>addEmployee()</code>	$O(1)$	$O(1)$	$O(1)$	$O(1)$
<code>searchEmployee()</code>	$O(1)$	$O(n/2)$	$O(n)$	$O(1)$
<code>traverseAll()</code>	$O(n)$	$O(n)$	$O(n)$	$O(1)$
<code>deleteEmployee()</code>	$O(1)$	$O(n/2)$	$O(n)$	$O(1)$

► Output Screenshot

```

Exercise 4 - Output Terminal

=====
Exercise 4: Employee Management System
=====

--- Adding Employees ---
Added: Employee[ID=201, Name=Ravi Kumar      , Position=Software Engineer, Salary=85000.00]
Added: Employee[ID=202, Name=Priya Sharma   , Position=Data Analyst      , Salary=72000.00]
Added: Employee[ID=203, Name=Arjun Singh    , Position=Project Manager   , Salary=110000.00]
Added: Employee[ID=204, Name=Neha Patel     , Position=QA Engineer       , Salary=65000.00]
Added: Employee[ID=205, Name=Vikram Rao     , Position=DevOps Engineer   , Salary=90000.00]

--- All Employees ---
Employee[ID=201, Name=Ravi Kumar      , Position=Software Engineer, Salary=85000.00]
Employee[ID=202, Name=Priya Sharma   , Position=Data Analyst      , Salary=72000.00]
Employee[ID=203, Name=Arjun Singh    , Position=Project Manager   , Salary=110000.00]
Total: 5 employees

--- Search Employee by ID ---
Search ID=203: Employee[ID=203, Name=Arjun Singh, Position=Project Manager, Salary=110000.00]
Search ID=999: Not Found

--- Delete Employee ---
Deleting: Employee[ID=202, Name=Priya Sharma, Position=Data Analyst, Salary=72000.00]
Employee ID 999 not found.

--- All Employees (After Delete) ---
Employee[ID=201, Name=Ravi Kumar      , Position=Software Engineer, Salary=85000.00]
Employee[ID=203, Name=Arjun Singh    , Position=Project Manager   , Salary=110000.00]
Employee[ID=204, Name=Neha Patel     , Position=QA Engineer       , Salary=65000.00]
Employee[ID=205, Name=Vikram Rao     , Position=DevOps Engineer   , Salary=90000.00]
Total: 4 employees

--- Time Complexity Analysis ---
Add      : O(1) if capacity available
Search   : O(n) - linear scan
Traverse : O(n)   Delete   : O(n) - scan + shift

```

Exercise 5: Task Management System

► Types of Linked Lists

- Singly Linked List: Each node has data + pointer to next node. Traversal only forward. $O(1)$ insert at head.
- Doubly Linked List: Each node has data + pointer to next AND previous. $O(1)$ insert/delete at both ends, but uses more memory.
- Circular Linked List: Last node points back to head. Useful for round-robin scheduling.

► Java Implementation (Singly Linked List)

```
class Task {
    private int taskId;
    private String taskName;
    private String status;
    Task next; // link to next node

    public Task(int taskId, String taskName, String status) {
        this.taskId = taskId;
        this.taskName = taskName;
        this.status = status;
        this.next = null;
    }
}

class TaskLinkedList {
    private Task head;
    private int size;

    // Add at end - O(n)
    public void addTask(Task task) {
        if (head == null) { head = task; }
        else {
            Task curr = head;
            while (curr.next != null) curr = curr.next;
            curr.next = task;
        }
        size++;
    }

    // Add at front - O(1)
    public void addTaskAtFront(Task task) {
        task.next = head;
        head = task;
        size++;
    }

    // Search - O(n)
    public Task searchTask(int taskId) {
        Task curr = head;
        while (curr != null) {
            if (curr.getTaskId() == taskId) return curr;
            curr = curr.next;
        }
    }
}
```

```

    }
    return null;
}

// Delete - O(n)
public boolean deleteTask(int taskId) {
    if (head == null) return false;
    if (head.getTaskId() == taskId) { head = head.next; size--; return true; }
    Task prev = head, curr = head.next;
    while (curr != null) {
        if (curr.getTaskId() == taskId) {
            prev.next = curr.next; size--; return true;
        }
        prev = curr; curr = curr.next;
    }
    return false;
}
}

```

► Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
addTask() at end	$O(n)$	$O(n)$	$O(n)$	$O(1)$
addTaskAtFront()	$O(1)$	$O(1)$	$O(1)$	$O(1)$
searchTask()	$O(1)$	$O(n)$	$O(n)$	$O(1)$
traverseAll()	$O(n)$	$O(n)$	$O(n)$	$O(1)$
deleteTask()	$O(1)$	$O(n)$	$O(n)$	$O(1)$

Advantage over Arrays: Dynamic size (no reallocation), $O(1)$ front insertion, no wasted pre-allocated memory. Disadvantage: No random index access, $O(n)$ traversal required.

► Output Screenshot

```

Exercise 5 – Output Terminal

=====
Exercise 5: Task Management System
=====

--- Adding Tasks ---
Added: Task[ID=301, Name=Design Database Schema, Status=Pending]
Added: Task[ID=302, Name=Develop REST API, Status=In Progress]
Added: Task[ID=303, Name=Write Unit Tests, Status=Pending]
Added: Task[ID=304, Name=Deploy to Staging, Status=Pending]
Added: Task[ID=305, Name=Code Review, Status=Completed]
Added at front: Task[ID=300, Name=Setup Environment, Status=Completed]

--- Task List (Linked List) ---
1. Task[ID=300, Name=Setup Environment, Status=Completed]
2. Task[ID=301, Name=Design Database Schema, Status=Pending]
3. Task[ID=302, Name=Develop REST API, Status=In Progress]
4. Task[ID=303, Name=Write Unit Tests, Status=Pending]
5. Task[ID=304, Name=Deploy to Staging, Status=Pending]
6. Task[ID=305, Name=Code Review, Status=Completed]
Total: 6 tasks

--- Search Task by ID ---
Search ID=303: Task[ID=303, Name=Write Unit Tests, Status=Pending]
Status Updated: Task[ID=303, Name=Write Unit Tests, Status=In Progress]

--- Delete Task ---
Deleting: Task[ID=300, Name=Setup Environment, Status=Completed]
Deleting: Task[ID=304, Name=Deploy to Staging, Status=Pending]
Task ID 999 not found.

--- Time Complexity ---
Add (front): O(1) Add (end): O(n)
Search: O(n) Traverse: O(n) Delete: O(n)
Advantage: Dynamic size, no pre-allocated memory waste.

```

Exercise 6: Library Management System

► Search Algorithm Theory

Linear Search scans each element sequentially until the target is found or the list ends. Binary Search works on sorted arrays by repeatedly halving the search space.

- Linear Search: No prerequisite — works on unsorted data. Each element compared once.
- Binary Search: Requires sorted array. Eliminates half the remaining elements each iteration.

► Java Implementation

```
class Book {
    private int bookId;
    private String title;
    private String author;
    // constructor + getters
}

// Linear Search by title - O(n)
public static Book linearSearchByTitle(Book[] books, String title) {
    for (Book b : books)
        if (b.getTitle().equalsIgnoreCase(title)) return b;
    return null;
}

// Linear Search by author - O(n)
public static Book linearSearchByAuthor(Book[] books, String author) {
    for (Book b : books)
        if (b.getAuthor().equalsIgnoreCase(author)) return b;
    return null;
}

// Binary Search by title - O(log n), array must be sorted
public static Book binarySearchByTitle(Book[] sorted, String title) {
    int low = 0, high = sorted.length - 1;
    while (low <= high) {
        int mid = (low + high) / 2;
        int cmp = sorted[mid].getTitle().compareToIgnoreCase(title);
        if (cmp == 0) return sorted[mid];
        else if (cmp < 0) low = mid + 1;
        else high = mid - 1;
    }
    return null;
}
```

► Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
Linear (Title)	$O(1)$	$O(n)$	$O(n)$	$O(1)$

Linear (Author)	$O(1)$	$O(n)$	$O(n)$	$O(1)$
Binary (Title)	$O(1)$	$O(\log n)$	$O(\log n)$	$O(1)$
Sort (Arrays.sort)	–	$O(n \log n)$	$O(n \log n)$	$O(\log n)$

Decision Guide: Use Linear Search for small catalogs (<1000 books) or searching unsorted/multi-field data. Use Binary Search for large title-indexed catalogs where data is pre-sorted.

► Output Screenshot

```

Exercise 6 – Output Terminal

=====
Exercise 6: Library Management System
=====

Library Catalog:
Book[ID=401, Title=The Pragmatic Programmer , Author=David Thomas]
Book[ID=402, Title=Clean Code , Author=Robert C. Martin]
Book[ID=403, Title=Introduction to Algorithms, Author=Thomas H. Cormen]
Book[ID=404, Title=Design Patterns , Author=Gang of Four]
Book[ID=405, Title=Artificial Intelligence , Author=Stuart Russell]

--- Linear Search by Title ---
Searching: "Clean Code"
Result : Book[ID=402, Title=Clean Code, Author=Robert C. Martin]
Time : 1801 ns

--- Linear Search by Author ---
Author: "Andrew S. Tanenbaum"
Result: Book[ID=407, Title=Computer Networks, Author=Andrew S. Tanenbaum]

--- Binary Search by Title (Sorted) ---
Searching: "Clean Code"
Result : Book[ID=402, Title=Clean Code, Author=Robert C. Martin]
Time : 2360 ns

--- Searching Non-existent Book ---
Linear: None Binary: None

--- Algorithm Comparison ---
Algorithm | Complexity | Requires Sorting?
-----|-----|-----
Linear Search |  $O(n)$  | No
Binary Search |  $O(\log n)$  | Yes

Use Linear : Small/unsorted datasets, search by any field.
Use Binary : Large sorted datasets for fast retrieval.

```

Exercise 7: Financial Forecasting

► Concept of Recursion

Recursion is a technique where a function calls itself with a smaller sub-problem until a base case is reached. It elegantly models problems that have a self-similar structure.

- **Base Case:** The condition that stops recursion (prevents infinite loop). Example: `years == 0`.
- **Recursive Case:** The function calls itself with a reduced problem. Example: $FV(n) = FV(n-1) \times (1+rate)$.
- **Call Stack:** Each recursive call uses stack memory; deep recursion risks `StackOverflowError`.
- **Memoization:** Caching sub-results to avoid redundant computation — converts $O(2^n)$ to $O(n)$.

► Java Implementation

```
// Recursive Future Value:  $FV(n) = principal \times (1 + rate)^n$ 
// Time:  $O(n)$  | Space:  $O(n)$  call stack
public static double futureValueRecursive(double principal, double rate, int years) {
    if (years == 0) return principal; // base case
    return futureValueRecursive(principal, rate, years - 1) * (1 + rate);
}

// Iterative equivalent -  $O(n)$  time,  $O(1)$  space
public static double futureValueIterative(double principal, double rate, int years) {
    double value = principal;
    for (int i = 0; i < years; i++) value *= (1 + rate);
    return value;
}

// Memoized version - avoids redundant computation
private static Map<Integer, Double> memo = new HashMap<>();

public static double futureValueMemo(double principal, double rate, int years) {
    if (years == 0) return principal;
    if (memo.containsKey(years)) return memo.get(years); // cache hit
    double result = futureValueMemo(principal, rate, years - 1) * (1 + rate);
    memo.put(years, result); // store result
    return result;
}

// Fibonacci growth model - models non-linear compounding
public static long fibGrowth(int n, long[] dp) {
    if (n <= 1) return n;
    if (dp[n] != -1) return dp[n];
    dp[n] = fibGrowth(n-1, dp) + fibGrowth(n-2, dp);
    return dp[n];
}
```

► Time Complexity Analysis

Operation	Best Case	Average Case	Worst Case	Space
-----------	-----------	--------------	------------	-------

Recursive (naive)	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Iterative	$O(n)$	$O(n)$	$O(n)$	$O(1)$
Memoized Recursive	$O(n)$	$O(n)$	$O(n)$	$O(n)$
Fib (naive)	$O(1)$	$O(2^n)$	$O(2^n)$	$O(n)$
Fib (memoized)	$O(1)$	$O(n)$	$O(n)$	$O(n)$

Optimization Strategy: Pure recursion risks stack overflow for large inputs (years > 10,000). For production financial systems, use iterative approach ($O(1)$ space). Apply memoization when multiple overlapping sub-problems exist, such as in Fibonacci-based growth models.

► Output Screenshot

```

Exercise 7 – Output Terminal

=====
Exercise 7: Financial Forecasting
=====

Principal : Rs 100,000.00      Growth Rate: 8% per year

--- Future Value Forecast (Recursive vs Iterative) ---
Year  | Recursive (Rs) | Iterative (Rs) | Match?
-----|-----|-----|-----
1     | 108000.00      | 108000.00      | YES
2     | 116640.00      | 116640.00      | YES
5     | 146932.81      | 146932.81      | YES
10    | 215892.50      | 215892.50      | YES
15    | 317216.91      | 317216.91      | YES
20    | 466095.71      | 466095.71      | YES

--- Memoized Recursive (Optimized) ---
Year 5 : Rs 146,932.81 [Time: 2583 ns]
Year 10 : Rs 215,892.50 [Time: 2303 ns]
Year 20 : Rs 466,095.71 [Time: 3037 ns]
Year 30 : Rs 1,006,265.69 [Time: 6862 ns]

--- Fibonacci Growth Model (Memoized) ---
Period 1: Fib=1   Period 2: Fib=1   Period 3: Fib=2
Period 4: Fib=3   Period 5: Fib=5   Period 6: Fib=8
Period 7: Fib=13  Period 8: Fib=21  Period 9: Fib=34

--- Complexity Analysis ---
Recursive (naive) : O(n) time | O(n) space
Iterative         : O(n) time | O(1) space ← BEST
Memoized Recursive : O(n) time | O(n) space
Fib Recursive (naive) : O(2^n) time | O(n) space
Fib Memoized      : O(n) time | O(n) space

Optimization: Memoization converts O(2^n) Fib to O(n).
For production: Use iterative to avoid stack overflow.

```

Summary & Conclusions

This assignment demonstrated the practical application of core data structures and algorithms through real-world Java scenarios:

- Exercise 1 — HashMap enables $O(1)$ inventory operations, far superior to ArrayList's $O(n)$ search.
- Exercise 2 — Binary Search ($O(\log n)$) outperforms Linear Search ($O(n)$) for sorted product data.
- Exercise 3 — Quick Sort ($O(n \log n)$ avg) is dramatically faster than Bubble Sort ($O(n^2)$) for large order lists.
- Exercise 4 — Arrays provide $O(1)$ indexed access but $O(n)$ search/delete; best for fixed-size datasets.
- Exercise 5 — Singly Linked Lists offer $O(1)$ head insertion and dynamic sizing without pre-allocation overhead.
- Exercise 6 — Algorithm choice depends on dataset characteristics: sorted $\rightarrow$ Binary; unsorted $\rightarrow$ Linear.
- Exercise 7 — Memoization converts exponential $O(2^n)$ Fibonacci recursion to linear $O(n)$, enabling practical financial forecasting.

Key Takeaway: Choosing the right data structure and algorithm is not merely academic — it can mean the difference between a system that responds in milliseconds and one that times out under production load.

— End of Assignment —